

Benchmarking vLLM on XLA Backend

Measuring LLM serving across GPU (CUDA) and TPU (XLA) backends



Rabimba Karanjai, PayPal AI Lab

Staff MLE

Google Developer Expert · ML & Cloud
NVIDIA – Developers Champion

Why we started

The decision TPUs are now a credible serving option — more available, cheaper per chip, and in 2026 the vLLM + OpenXLA stack matured. Should we serve LLMs on TPU instead of GPU?

The problem We couldn't answer it responsibly — every public comparison changed the model, client, SLO, precision, or warmup state.

So we built the benchmark we wished existed — one harness, one variable, honest claims — and open-sourced it.

A real production decision — with no trustworthy way to make it.

The question production teams actually ask

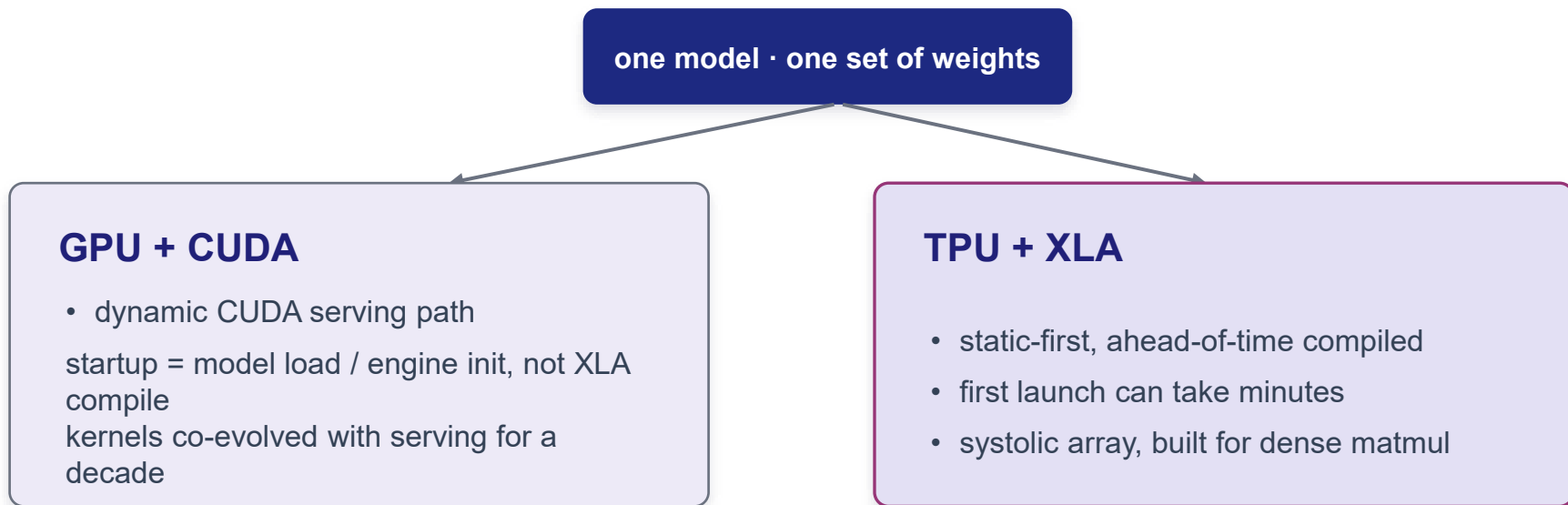
Common framing “which accelerator has the highest tokens / sec?” — a peak-throughput number.

What teams need for a given model & request mix: does the backend keep first-token (TTFT) and per-token (TPOT) latency within the SLO — at acceptable cost and operational risk?

Peak tokens / sec is not the same as usable capacity.

TTFT = time to first token · TPOT = time per output token · SLO = service-level objective (a latency target)

Why the backend matters: same weights, different behavior



Why it matters: identical weights, but the runtime, compiler, kernels and interconnect differ — so latency, throughput and startup differ. Parity is never a given; you have to measure it.

Why public benchmarks don't answer this.

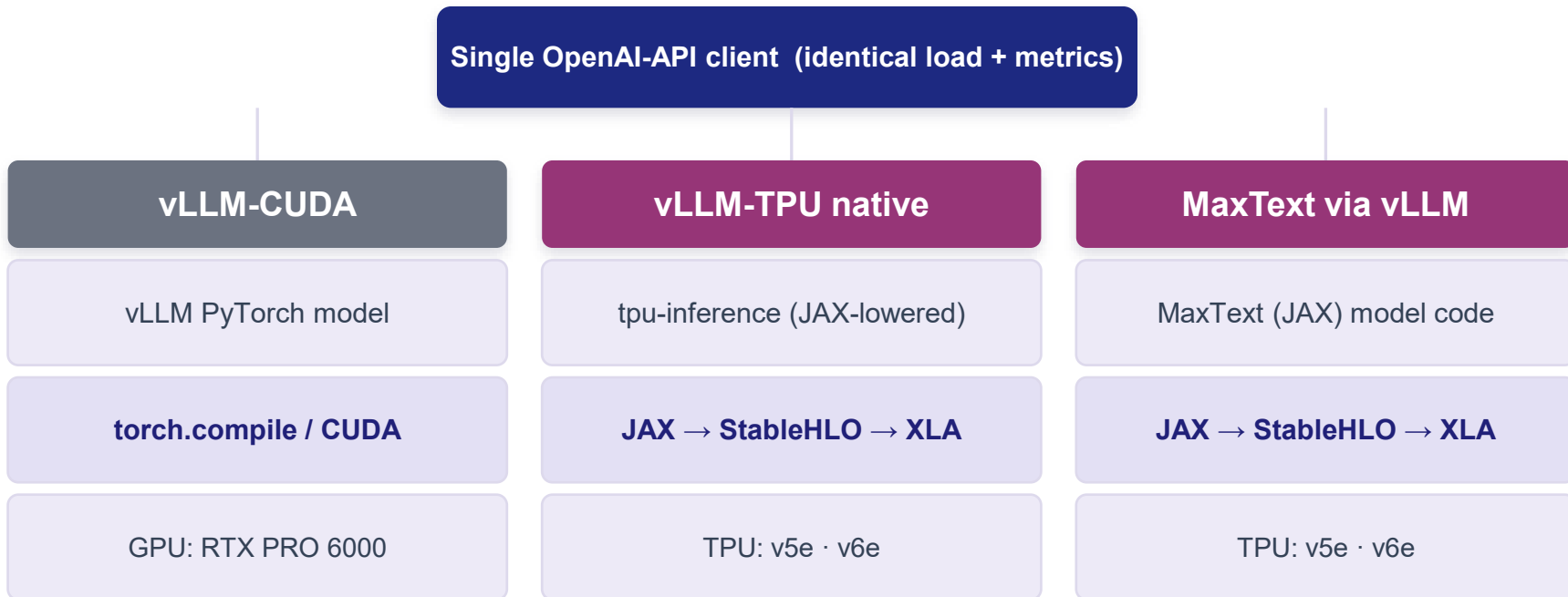
They show peak tokens / sec — often warm-only, on one shape.

They hide the client, SLO, precision, warmup and request mix — so the charts aren't comparable.

So you can't tell if a backend meets YOUR SLO — and the number is easy to game.

You can't crown a winner from numbers that were never measured the same way.

Benchmark design: only the server changes.



Held fixed: model · request mix · client + metric math · SLO · token accounting (cold+warm). Only the server changes — so any delta is the backend.

Our testbed — what we measured vs. what we reference

MEASURED · our exact stack

Serving vLLM 0.13.0 · tpu-inference path off (CUDA)

Framework PyTorch 2.9.0+cu128 · Transformers 4.57.6

Driver/CUDA CUDA 13.0 · NVIDIA driver 580.95.05

Hardware 8× RTX PRO 6000 Blackwell · PCIe Gen5, no NVLink

Models Llama-3.1 8B & 70B · bf16

Client one OpenAI-API streaming client

REFERENCED · public, not co-measured

Stack vLLM-TPU / tpu-inference · MaxText / JetStream

Hardware TPU v5e · v6e (Trillium)

Source Google Cloud AI Hypercomputer, Apr 2025

Role a yardstick only — never plotted as ours

PLANNED run TPU v6e through the same client, models, SLO & one pricing basis — then we can claim goodput-per-dollar.

The four metrics we report — and why

1 TTFT — time to first token
responsiveness: how long until the user sees the first token.

2 TPOT — time per output token
streaming speed: the gap between tokens once generating.

3 Throughput — tokens / sec
raw output capacity the server sustains.

4 Goodput — good requests / sec
requests/sec that meet the SLO — the capacity you can actually use.

Latency (TTFT, TPOT) tells you if it's fast enough; capacity (throughput, goodput) tells you how much load it holds.

TTFT = time to first token · TPOT = time per output token · SLO = service-level objective (a latency target)

Each request has two phases: prefill and decode

PREFILL

process the whole prompt at once — compute-bound. Sets TTFT (time to first token).



DECODE

generate one token at a time — memory-bandwidth-bound. Sets TPOT (time per output token).



Two phases, two bottlenecks — so prefill-heavy and decode-heavy workloads stress different parts of the chip.

Throughput vs. goodput: which capacity counts

Throughput

total tokens/sec — but it also counts requests that already missed their latency target.



GOODPUT

requests/sec meeting the TTFT and TPOT targets at once — the capacity you can actually serve.

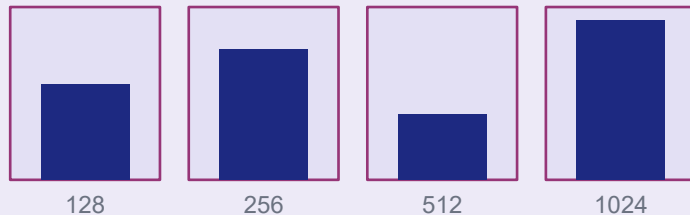


So we rank backends by goodput at a fixed SLO: throughput can look healthy while the service is already failing users.

XLA compiles per shape; serving shapes keep changing

XLA (the TPU compiler) builds a separate program per tensor shape.

requests → padded to precompiled buckets



⚡ Serving has many shapes (varying prompt & output lengths). XLA pads each to a precompiled bucket — wasted compute — and pays a one-time first-compile cost. That cold path is what GPUs mostly avoid.

Why these workloads — each isolates a bottleneck

Workload (input → output)	What it is	Bottleneck it isolates
ShareGPT — real (~220 → 280)	real chat traffic	mixed prefill + decode
Synthetic 1024 → 128	long prompt, short reply	prefill · compute-bound (TTFT)
Synthetic 256 → 2048	short prompt, long reply	decode · bandwidth-bound (TPOT)

Models: Llama-3.1 8B (fits one chip) · 70B (needs tensor parallelism → exposes the interconnect).

Takeaway: because the load sweeps prefill→decode and 8B→70B, every result maps to one bottleneck — so when a backend wins or breaks, we know which one.

How we measure each metric



The design

One streaming client drives every backend; only the server changes.

vLLM bench-serve semantics, zero vLLM dependency.

TTFT = time of the first streamed token. TPOT = (end-to-end - TTFT) / (output tokens - 1).

Token counts come from the API usage field — exact, not estimated by counting chunks.

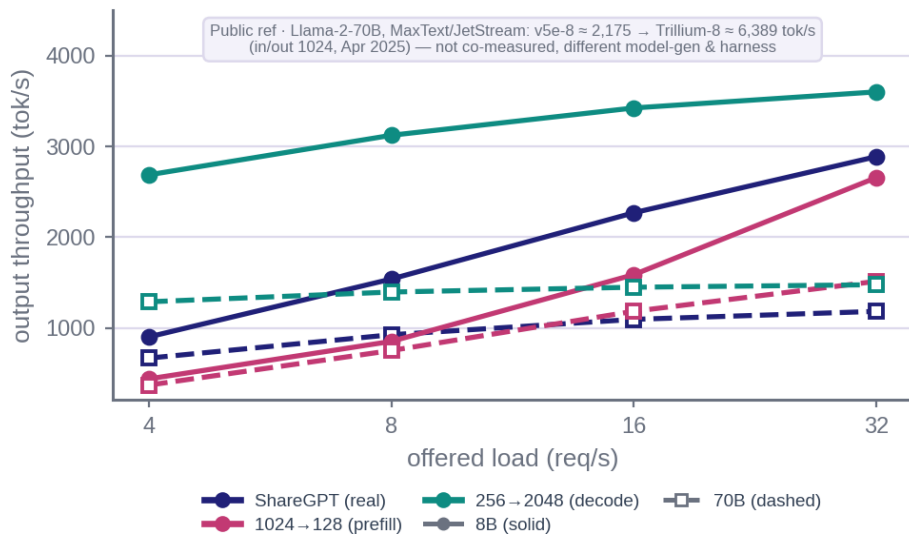
Poisson arrivals at a target rate — realistic, bursty load.

Warmup runs separately; the first request measures the cold-start path.

ignore_eos fixes the output length so token counts are comparable across backends.

Throughput vs. load — the system looks healthy

Measured · RTX PRO 6000 (CUDA) · Llama-3.1 8B & 70B, bf16 · 70B = multi-GPU TP≥2 over PCIe (no NVLink)



Throughput alone is the trap

Tokens/sec rises with load — 8B to ~5,131, 70B to ~2,187 — so both look healthy here. But throughput counts every token, even from requests that already missed their SLO. The next two slides show what it hides.

Public TPU ref (not co-measured): Google Cloud AI Hypercomputer, Apr 2025 — MaxText/JetStream, Fig.1. Osmos: >3,500 tok/s/node is a separate production quote.

Tail latency — 70B exceeds the TPOT target

Measured · RTX PRO 6000 (CUDA) · Llama-3.1 8B & 70B, bf16 · 70B = multi-GPU TP≥2 over PCIe (no NVLink)

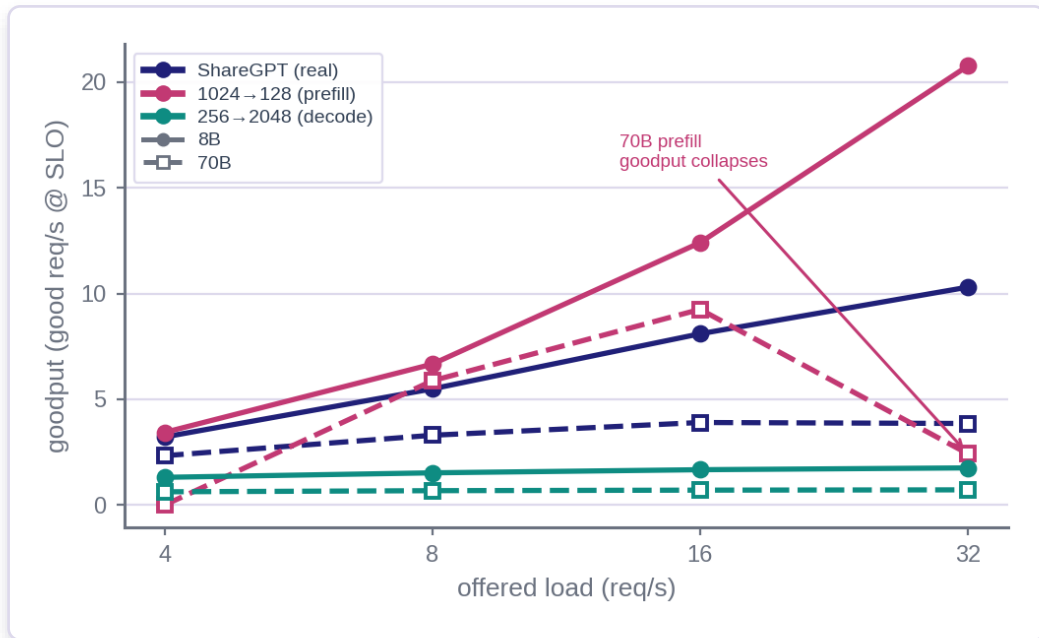


The tail is where 70B breaks

At P99, 8B holds TTFT ≤ 328 ms and TPOT ≤ 38 ms — under SLO everywhere. 70B's per-token tail (TPOT) passes the 50 ms target (up to ~466): split across chips, PCIe traffic stretches decode. Throughput looked fine — the tail breaks first.

Goodput — the capacity that meets the SLO

Measured · RTX PRO 6000 (CUDA) · Llama-3.1 8B & 70B, bf16 · 70B = multi-GPU TP≥2 over PCIe (no NVLink)

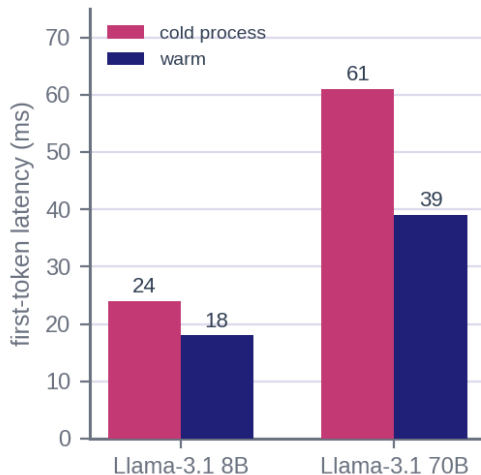


Goodput is the honest number

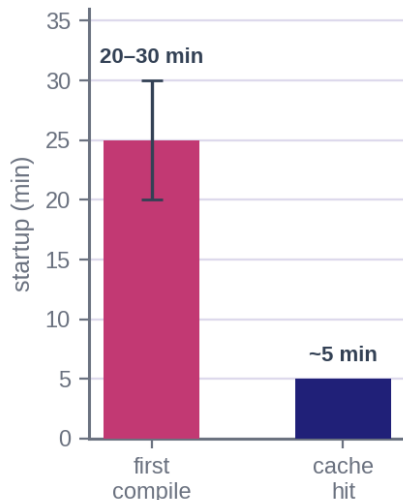
Goodput counts only requests meeting both SLOs. 8B's tracks the load — real usable capacity. 70B's collapses (9.3 → 2.4 good req/s) as its decode tail misses target. Same runs as the throughput chart — now the gap is unmissable.

Startup cost — GPU is instant, TPU compiles first

GPU · measured (no compile wall)



TPU/XLA · public (one-time)



A cost steady-state hides

GPU first-token is the same cold or warm (24–61 ms) — no compile wall. TPU/XLA pays 20–30 min on the first compile (~5 min cached). Throughput charts never show this, but it sets how fast you scale up or recover on spot.

Public TPU compile: older vLLM TPU docs ≈ 20–30 min first / ~5 min cached; current docs report minutes-to-~1 hr by model & context. Not co-measured.

The takeaway: how to benchmark serving fairly

- 1 **Fix the client** one streaming client, usage-based tokens, identical metric math.
- 2 **Fix SLO & workload** TTFT / TPOT targets + a prefill→decode ladder, held constant.
- 3 **Measure cold AND warm** the compile / warmup path is a real deployment cost.
- 4 **Rank by goodput** at a fixed SLO — then publish artifacts + claim boundary.

The takeaway isn't "GPU wins" or "TPU wins" — it's the contract.

Thank **you!**

Learn more

Code: github.com/rabimba/vllm-xla-bench

Event: openxla.org/events/summer_devlab_2026

Rabimba Karanjai · PayPal AI Lab · Google Developer Expert
rkaranjai@paypal.com



Connect on LinkedIn

linkedin.com/in/rabimba



Why serving and XLA collide (detail)

Dynamic, ragged serving

- variable sequence lengths
- bursty arrivals
- irregular MoE routing



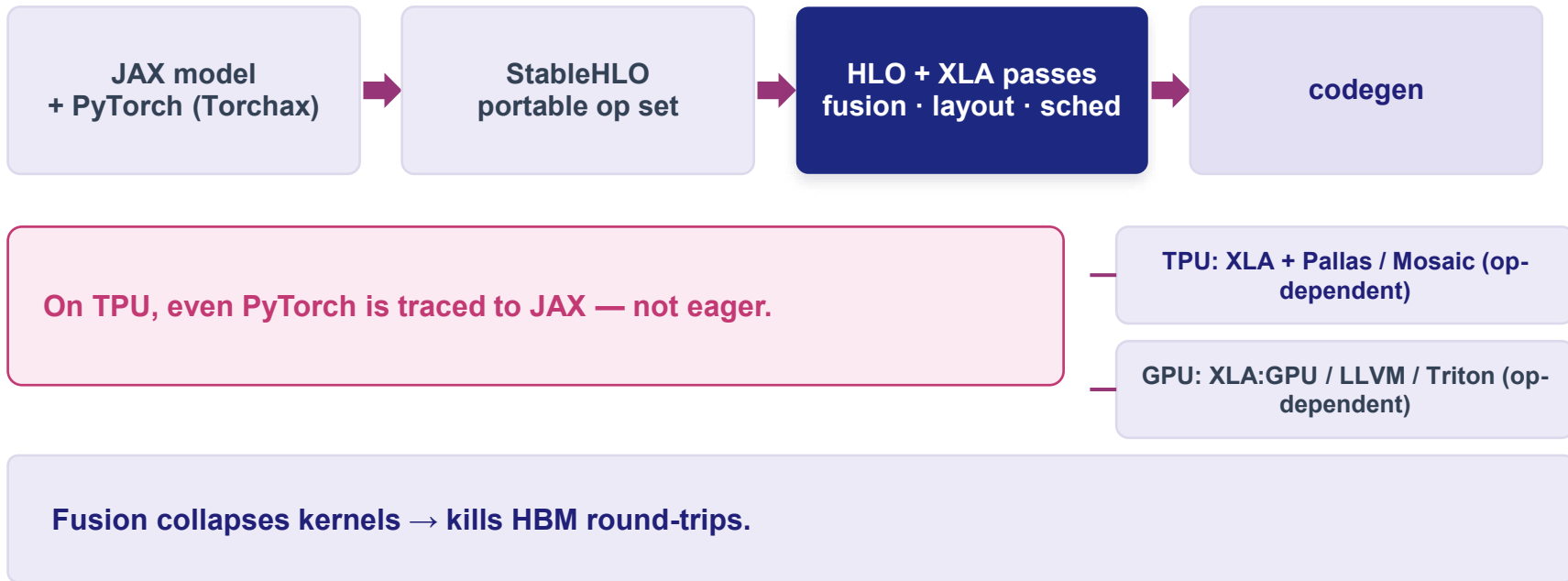
impedance
mismatch

Static-first XLA

- compiles for fixed shapes
- ahead-of-time, cached
- loves dense, regular matmul

Where that mismatch helps, hurts, or washes out.

How the model gets compiled (the XLA pipeline)



Large models span chips — limited by the interconnect



How sharding works

70B doesn't fit on one chip, so it's split across chips (tensor parallelism, TP). Every layer then needs a cross-chip all-reduce — so throughput is bounded by the interconnect fabric, not raw compute.

TPU ICI mesh

high-BW torus, built for collectives

RTX PRO 6000

NO NVLink → PCIe Gen5 — collectives bottlenecked

Why it's here: this predicts the 70B result. The RTX PRO 6000 has no NVLink, so at $TP > 1$ collectives fall back to PCIe — the bottleneck you'll see in the latency tail.

Keeping the benchmark defensible

1

bf16 everywhere

apples-to-apples cell; fp8/fp4/int8 only in labeled separate cells

2

One identical client

same load gen + metric math; never a confound

3

Fixed SLO per class

held identical across backends — goodput as a single yardstick

4

Cold AND warm

cache cleared to force a true cold path; never discard compilation

5

Full env capture

vLLM / JAX / libtpu / XLA build, flags, seeds, SKUs — artifact norms

6

Effort asymmetry stated

vLLM-CUDA is more mature — named next to every cost number

The 2026 vLLM + TPU serving stack

≤ 2025 · three engines

JetStream

vLLM-TPU

vLLM-CUDA



2026 · one frontend + one JAX-native backend

vLLM — frontend (scheduler · OpenAI API · PyTorch + JAX)

tpu-inference — JAX-native backend

model code — tpu-inference optimized when available; else vLLM PyTorch via Torchax

JetStream archived Feb 2026 → folded into tpu-inference.

Ragged Paged Attention (RPA v3)

GPU

mature, hand-tuned CUDA

- FlashAttention · PagedAttention · years of tuning

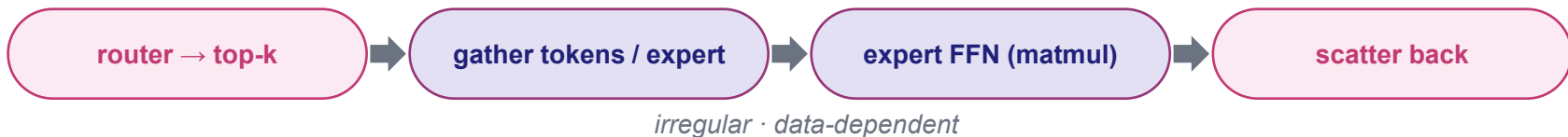
TPU — Ragged Paged Attention v3

Pallas + Mosaic

- Ragged tiling; fuses KV scatter into attention

RPA: up to 86% MBU on decode — the primary path.

MoE — a future workload (not measured)



GPU streaming multiprocessors

- Flexible scheduling; mature MoE kernels

TPU MXU (systolic array)

- Systolic array loves dense matmul; routing is harder

Future workload — not measured here. Dense results may not generalize to MoE; does MoE's edge flip on TPU?

Exact commands (how we ran it)

1 · serve (GPU)

```
vllm serve meta-llama/Llama-3.1-8B-Instruct --tensor-parallel-size 1 \  
--max-model-len 32768 --dtype bfloat16 --port 8000
```

2 · serve (TPU native)

```
VLLM_XLA_CACHE_PATH=~/.cache/vllm/xla vllm serve meta-llama/Llama-3.1-70B-Instruct \  
--tensor-parallel-size 8 --max-model-len 32768 --port 8000
```

3 · sweep

```
python run_matrix.py --base-url http://127.0.0.1:8000 \  
--model meta-llama/Llama-3.1-8B-Instruct --backend-label vllm-cuda \  
--hardware-label rtx-pro-6000 --rates 1,2,4,8,16,32,inf
```

4 · cold-start

```
rm -rf ~/.cache/vllm/xla # force COLD  
python measure_startup.py --shapes 128:128,1024:128
```

5 · figures

```
python analyze.py --results results --pricing config/pricing.yaml --out out
```

Artifact manifest — exactly what we ran

ENVIRONMENT

- vLLM 0.13.0 · PyTorch 2.9.0+cu128
- Transformers 4.57.6 · NumPy 2.1.3
- CUDA 13.0 · NVIDIA driver 580.95.05
- 8× RTX PRO 6000 Blackwell (97,887 MiB)
- PCIe Gen5, no NVLink — nvidia-smi topo -m: pin
- host g4-8card-shared

SERVING

- Llama-3.1-8B (TP=1) · 70B
- dtype bf16 · max-model-len 32768
- max-num-seqs / max-num-batched-tokens: pin
- KV-cache dtype · tokenizer revision: pin

WORKLOAD

- ShareGPT (real) · synth 1024→128 · synth 256→2048
- request rates 4 / 8 / 16 / 32 + saturation
- seed 0 · 100–200 prompts / run · ignore_eos
- SLOs: TTFT ≤ 1 s · TPOT ≤ 50 ms